A3: LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Study guide 2: the plan from here to 13 November (status: 16 October 2026)

Contents

0. What this document is **3**

0.1 The plan on one page . . . 3

0.2 Marks, with a new bug and without one . . . 4

0.3 Three claims . . . 5

0.4 Where we are on 16 October . . . 5

0.5 Who does what . . . 6

1. Where we stand and what is still missing **6**

2. Step 1: tune the grammar arm, then fuzz for hours **7**

3. Step 2: metamorphic oracles on derivation trees **7**

3.1 Why we need them . . . 7

3.2 Ternary Logic Partitioning (TLP) . . . 8

3.3 NoREC . . . 8

3.4 Equivalent-expression rewrites (our unique twist) . . . 8

3.5 Design (how it fits the code) . . . 9

4. Step 3: cross-version differential testing **9**

5. Step 4: commit-directed LLM fuzzing (Stage 3, the unique angle) **9**

5.1 Idea . . . 9

5.2 Diff coverage (our metric) . . . 10

5.3 Feedback-weighted grammar alternatives . . . 10

5.4 Why the validation gate matters . . . 10

5.5 Hot targets (all to verify against the git log and release notes) . . . 10

6. Smaller generator upgrades **11**

7. Triage: from candidate to accepted bug **12**

8. Evaluation upgrades **12**

8.1 Research questions 12

8.2 The re-injected bug benchmark (ground truth) 12

8.3 Statistics 13

8.4 What the report must contain 13

9. Course lectures used (for the report and the viva) **14**

10. Timeline and owners **14**

11. Risks and fallbacks **15**

12. References (all: verify before citing) **16**

13. The next concrete step **16**

0. What this document is

Document 1 explains what exists. This document explains **what we will add, in what order, and why each addition raises either our marks or our chance of finding a real, upstream-accepted PostgreSQL bug**. Nothing here replaces the original proposal: the three-way comparison

```
Random/Mutation   ->   Grammar-only   ->   Grammar + LLM
```

stays the backbone of the paper. Everything below is layered on top of it as (a) stronger bug detectors, (b) smarter generation, and (c) a stronger evaluation.

Conventions:

- **(you)** marks a step that you run on your machine and paste back to me. Fuzzing campaigns take hours, so they must run on your machine.
- **(verify)** marks a fact about PostgreSQL or the literature that we must check against the git log or the paper itself before we write it in the report. Citations are listed in Section 12 and all of them carry this marker.
- Numbers in this document are **targets**, not results, except Section 0.4, which repeats measurements from Study guide 1.

0.1 The plan on one page

Priority	Addition	Why it matters
1	Get the Grammar arm running for hours (tuning loop)	Nothing else matters without volume
2	Metamorphic oracles on derivation trees (TLP, NoREC, rewrites)	Most new logic bugs in DBMSs are found this way
3	Cross-version differential (REL_19 vs master)	Finds regressions in brand-new code, which upstream fixes fastest
4	Triage pipeline (reduce, bisect, dedup, report)	Turns candidates into accepted bugs
5	Commit-directed LLM loop with diff coverage	The project's unique angle; the LLM's value is measured, not claimed
6	Re-injected bug benchmark + statistics	A complete, measured result that does not depend on finding a new bug
7	Stretch: plan-shape feedback, CERT-style performance oracle, RLS leak oracle	Extra depth for the paper

The order is deliberate: items 1 to 4 maximise the chance of a real bug early (we need time for upstream to respond before 13 November); items 5 and 6 are the paper, whether or not a new bug appears; item 7 only if time remains.

0.2 Marks, with a new bug and without one

Component	Weight	What we will actually hand in
Implementation and results	50 %	A working three-arm fuzzer, the oracles, the campaigns, and the triage pipeline. The course gives the **top** of this component only for a previously unknown bug the developers accept: a fix commit, or an acknowledgement on pgsql-bugs or pgsql-hackers. An acknowledgement is the target we can hit before 13 November; a merged fix is the best case. We file in week 2 so there is time for a reply.
Meetings and the final presentation	25 %	Every member can explain the grammar, the oracle, the statistics and this plan. These two study guides are the script. This component does not require a new bug.
Report, ACM/FSE format	25 %	Five graded parts: motivating example, workflow, research questions, related work, threats to validity. Each one is filled with measured numbers. This component does not require a new bug.

The report is designed so a missing bug does not collapse it:

- **RQ1, RQ3 and RQ4 are complete results with zero new bugs.** RQ3 is the safety net: we re-inject historical bugs and measure time to detect. That number exists because we create the bugs, not because we get lucky.
- **The motivating example is guaranteed to be real.** The report template requires a real example with measured numbers. If we have a new bug, that example is the new bug. If we do not, it is a re-injected historical bug that the grammar arm detects and the random arm misses, with the times.
- **RQ2 stays honest.** Zero new bugs is a valid answer. We also report how many candidates each oracle produced and how many triage rejected (duplicates, non-determinism, not a bug). That is the oracle's precision, and it is a result either way.
- **We do not spend the 50 % in advance.** A strong paper does not replace an accepted bug on the implementation component. The schedule puts filing ahead of polishing figures for that reason.

0.3 Three claims

The three-arm comparison stays the backbone. These are the three things we want the professor to remember. The first two are partly in the code today; the third is Stage 3.

1. **One tree, four jobs.** The derivation tree is the only representation. Generation builds it. Mutation edits a subtree and replays, re-binding names and types. Metamorphic testing rewrites one typed node. Reduction replaces a subtree by the smallest alternative of the same type. The SQL text is only the rendering. A rewrite or a mutation cannot break types or scopes, which is why the metamorphic oracle can be aggressive without a pile of syntax errors. Generation and mutation exist; rewrite and reduction are the next code we write.

2. **Soundness we measure, instead of claiming it.**

   - The generator is *sound for its fragment* when every derivation is SQL the server accepts, and *incomplete on purpose*: we do not cover all of SQL. The valid-statement rate is the measurement.
   - The oracle is *sound modulo the determinism filter* when a reported mismatch is a real disagreement between two executions, and *incomplete* because many bugs cause no mismatch. Triage precision (real bugs divided by reports) is the measurement. Saying both out loud is what stops the project sounding like "our fuzzer finds all the bugs".

3. **The LLM proposes; the server decides.** No LLM text enters the fuzzer, and no LLM text becomes a number in the paper, unless the grammar parses, the server runs the statements, and the changed lines of that commit gain coverage. Diff coverage is the proof that the LLM helped. Token cost is reported next to it. This also matches the course rule that every number comes from a real run.

Localization, using oracles we already run. On one query we run the 17-variant plan oracle and one metamorphic rewrite. The pair says where the bug is, which is what makes a report get a reply:

- only the plan oracle fails: a planner pass, and the setting that diverges names it;
- only the rewrite fails: a transformation that the planner settings do not toggle;
- both fail: executor or expression evaluation.

Same campaign, sharper reports. It is not a new arm and not a new tool.

0.4 Where we are on 16 October

Priority 1 has a measurement. Four rounds of 300 grammar test cases, seed 1, on the assert-enabled server (full table in Study guide 1, Section 9.1):

Round	Accepted	Semantic errors	Time	Bug candidates
1	83.4% of 812	76	16.7 s	0
2	83.1% of 828	39	82.9 s	0
3	88.4% of 929	28	56.1 s	0
4	**100% of 966**	0	22.1 s	0

The random/mutation generator from Stage 1 is in `src/generator/` and is the baseline named in `configs/stage1.yaml`. It builds a small SQL AST, renders it, then mutates the text. It does not use the grammar, and it ignores feedback. That is the comparison the grammar has to win.

The next run, when we start the long campaigns, is two grammar trials of four hours:

```
cd src
python -m pgfuzz run configs/hunt.yaml -j 2 --no-plots
```

Each candidate is written under `results/hunt/grammar/trial-NN/failures/`.

0.5 Who does what

Venu, Bhuvan, and Gowtham. The earlier split (harness, generators, evaluation) is how the code was built. From this meeting on, the work is:

Person	Job
Venu	Recent PostgreSQL features. Read commits since the last release and extend the grammar so MERGE, SQL/JSON, generated columns, partitions, and new planner rewrites are actually executed.
Bhuvan	The same feature work, and the first pass on every bug candidate a campaign prints: reproduce, shrink, decide.
Gowtham	The rest of the project: the three-arm experiment, the statistics, the LLM arm, and the paper.

Two people on new code, because that is where an unknown bug is most likely. One person on candidates, so a find is not left in a log. One person on the paper, so the grade does not wait for a crash. The servers stay up during all three.

1. Where we stand and what is still missing

Already working (see Document 1): harness on a PostgreSQL `20devel` build with assertions (commit `d733fca`), the oracle (crashes, assertion failures, internal errors, timeouts), the 17-variant differential plan oracle, the typed `.pgg` grammar with guided-replay mutation, the `sample` command, the Stage 1 random/mutation generator in `src/generator/`, and 84 passing tests (65 before that generator was added).

Missing, in order of urgency:

1. **Server-side validity of the grammar arm** is now measured: 100% of 966 statements on the fourth 300-case sample, after 83.4%, 83.1%, and 88.4% (Section 0.4). The tuning loop in Section 2 is how we got there, and we will use it again when new rules land.
2. **No long campaign** has run yet, so we have no bug candidates.

3. **No metamorphic oracle**: the differential plan oracle only catches bugs that change with planner settings. Bugs in expression evaluation, in rewrites that are always applied, or in the executor are invisible to it.
4. **No triage automation**: reduction, bisecting and dedup are manual.
5. **The LLM arm (Stage 3) is not built.**
6. **No ground truth** for the evaluation if we find few bugs.

2. Step 1: tune the grammar arm, then fuzz for hours

This is the critical path. The loop is:

```
(you) sample --execute  ->  (me) fix worst alternatives  ->  repeat until valid >= 85 %
(you) run hunt.yaml for 4 h  ->  (me) triage candidates  ->  (me) tune, add oracles
```

The first line of that loop is done. Four samples, seed 1, reached 100% accepted on the last one: 966 of 966 statements, 22.1 seconds, no errors (Section 0.4). The second line, the four-hour hunt, has not been started.

Commands (you). From `src/`:

```
python -m pgfuzz -q sample configs/local.yaml -n 300 --execute `
    --seed 1 --out ../results/sample1
python -m pgfuzz run configs/hunt.yaml -j 2 --no-plots
```

Paste back: the printed summary (valid rate, top errors, worst alternatives). From a campaign: the final report table and the list of candidate directories.

Targets. At least 85 % of statements accepted by the server, at least 5 test cases per second per worker, no single error message above 5 % of all statements.

What I do with your output. Each top error points to one grammar alternative (the `worst_alternatives` statistic tells us which). I fix the rule, or lower its weight, or add a type restriction. Two or three rounds are usually enough.

> Once bug candidates appear, we stop treating them as a list and work on each one individually: reproduce, reduce, decide whether it is a real bug, bisect, and write it up. The strategy depends on the kind of candidate (crash, assertion, wrong result, internal error) and is described in Section 7. I will tell you exactly which commands to run for each one.

3. Step 2: metamorphic oracles on derivation trees

3.1 Why we need them

A **wrong-result bug** is the most valuable kind of DBMS bug (silent data corruption) and the hardest to detect, because there is no crash. Our differential plan oracle compares one query under 17 planner settings. It cannot see a bug that every plan shares. **Metamorphic testing** (course connection: test oracles, L13-L16) solves this: we transform the query into a different query whose result must be related in a known way, and check the relation.

3.2 Ternary Logic Partitioning (TLP)

For any predicate `p` and query `Q`:

$$Q \;=\; Q_{\text{WHERE } p} \;\uplus\; Q_{\text{WHERE NOT } p} \;\uplus\; Q_{\text{WHERE } p \text{ IS NULL}}$$

where $\uplus$ is multiset union (`UNION ALL`). SQL uses three-valued logic, so the three partitions cover every row exactly once. If the two sides differ, one of the queries is wrong. Variants exist for `DISTINCT` (use `UNION`), aggregates (`MIN`/`MAX` of the partition results; `SUM` of `SUM`s; `SUM` of `COUNT`s), `GROUP BY` and `HAVING`.

3.3 NoREC

The optimiser rewrites `SELECT count(*) FROM t WHERE p`, but it cannot rewrite much in `SELECT sum(CASE WHEN p THEN 1 ELSE 0 END) FROM t`, because the predicate moved into the select list. Both counts must be equal. This catches bugs in index selection, predicate push-down and partition pruning, which are exactly the subsystems that recent PostgreSQL releases changed.

3.4 Equivalent-expression rewrites (our unique twist)

Tools like SQLancer apply TLP and NoREC to **strings** they built themselves. Our generator has something better: a **typed derivation tree** for every query (Document 1, Section 6). Every node knows its grammar symbol, its SQL type, and which columns are in scope. So we can apply rewrites **exactly at the right node** and keep the query valid. Examples:

Rewrite	Must be equivalent	Targets (verify)
`x IN (a, b, c)` <-> `x = a OR x = b OR x = c`	yes (same NULL semantics)	OR-to-ANY transformation (PG18)
`p` -> `p AND c IS NOT NULL` for a NOT NULL column `c`	yes	NOT NULL based simplification (PG17/18)
`t JOIN s ON k` with unique `s.k` -> `WHERE EXISTS (...)`	yes for semi-join shape	join removal, self-join elimination
`WITH c AS (...)` -> `WITH c AS MATERIALIZED (...)`	yes	CTE inlining
`e` -> `CASE WHEN true THEN e END`	yes	constant folding
`e` -> `(SELECT e)` (scalar subquery)	yes	subquery pull-up
`a < b` -> `NOT (a >= b)` with NULL guard	yes under 3VL rules	predicate normalisation
`t` -> `(SELECT * FROM t) AS t`	yes	subquery flattening
`ORDER BY` removed when the result is compared as a multiset	yes	sort elimination, incremental sort

The equivalent-expression idea is published (EET, OSDI 2024 (verify)); doing it on a **typed, scope-aware grammar tree from a user-editable grammar** and letting the LLM add new rewrites is our contribution.

3.5 Design (how it fits the code)

```
GrammarGenerator ---> TestCase(+ derivation trees)
                              |
                        MetamorphicOracle
                          |  find eligible nodes (where clause, IN list, CTE, join ...)
                          |  build follow-up query by replace_at(tree, path, new_node)
                          |  run both inside the same savepoint, digest results
                          v
                    outcome = wrong_result (kind = tlp | norec | rewrite:<name>)
```

- Reuse the existing `eligible()` determinism check of the differential oracle (no `LIMIT`, no volatile functions, no order-sensitive aggregates) and the same order-insensitive digests and float masking.
- The signature of a metamorphic wrong result is `(rewrite name, plan node types of both queries)`, so the same bug found many times deduplicates.
- Apply at most 2 rewrites per query, chosen at random among eligible nodes, to keep throughput high.
- **False positive control**: before reporting, the oracle re-runs both queries, then runs them again with all planner variants switched off. If the difference disappears under every variant, it is reported as a planner bug; if it remains, it is reported as an executor or rewrite bug.

Effort. About 300 lines plus tests; I write it while your first campaign runs.

4. Step 3: cross-version differential testing

Build a second server from the `REL_19_STABLE` branch (the latest release line, (verify) branch name) and run every eligible query on both servers. If master returns a different multiset than the release, either master has a new bug (a **regression**, the most valuable kind of report because the code is still fresh in the committer's mind) or the behaviour change was intentional (documented in the release notes or the commit message; we filter these).

Why this has a high chance of finding a real bug:

- New features and optimisations land on master first and get the least testing.
- Upstream treats regressions on master as **open items** that must be fixed before the next release, so reports are acted on quickly.
- The release server is an independent reference implementation for free.

Cost: one more build (about 13 minutes on your machine) and one more port.

(you) I will give you a short script: build `REL_19_STABLE` into a second prefix, start it on port 5434, done.

5. Step 4: commit-directed LLM fuzzing (Stage 3, the unique angle)

5.1 Idea

Most bugs live in **recently changed code**. Directed greybox fuzzing (AFLGo, CCS 2017 (verify)) steers a fuzzer towards chosen code locations. We do the same at the level of **SQL features**, and we let the

LLM do the part that is hard for a program: understanding what a commit changed and which SQL reaches it.

```
git log REL_19 branch point..master  --  src/backend/optimizer, executor, parser, utils/adt
      |
      v
 commit digest  (message, diff summary, regression .sql test added by the commit)
      |
      v
 LLM  ->  proposes:  (a) new .pgg rules / weights   (b) seed test cases
      |
      v
 validation gate:  grammar parses,  200 samples,  server validity >= 80 %,
                   and new coverage of the changed lines  (else: reject + feedback to LLM)
      |
      v
 accepted patch enters the Grammar+LLM arm
```

5.2 Diff coverage (our metric)

For every commit, the changed lines in `src/backend/**` are the **targets**. Using gcov data from the coverage build, we measure for each arm:

$$\text{diff coverage} = \frac{|\text{changed lines executed}|}{|\text{changed executable lines}|}$$

This answers "does the LLM actually help?" with a number per commit. The Grammar-only arm is the control.

5.3 Feedback-weighted grammar alternatives

Each grammar alternative has a weight. The generator already records, per alternative, how often it produced valid SQL. We add a **multi-armed bandit** update: alternatives whose test cases hit new features, new plan shapes or changed lines get their weight increased; alternatives that produce mostly errors get it decreased. This is cheap (a few lines in `feedback()`) and connects to the "greybox" part of fuzzing (L15-16).

5.4 Why the validation gate matters

LLMs produce plausible but wrong SQL. Our gate means **no LLM output enters the fuzzer unless it parses, runs and adds coverage**. This is a verifier loop: the LLM is a proposal engine, the server and coverage are the judges. It also gives the paper a clean measurement: acceptance rate of LLM proposals, token cost, and coverage gained per accepted patch.

5.5 Hot targets (all to verify against the git log and release notes)

Area	What to stress	Grammar support today
Self-join elimination	`t JOIN t` on unique keys, with extra quals	`self_join` rule exists
OR-to-ANY transformation	long `OR` chains vs `IN` lists, mixed types, NULLs	partial
B-tree skip scan	multi-column indexes, predicates on non-leading column	indexes i0-i8
Virtual generated columns	in `WHERE`, indexes, partition keys, `UPDATE`	table `t4`
`RETURNING OLD/NEW`	in `UPDATE`, `DELETE`, `MERGE`	yes
`MERGE` (`BY SOURCE`, partitioned targets, views)	all action combinations	yes
SQL/JSON, `JSON_TABLE`	nested paths, `ON ERROR`, typed columns	yes
Grouping sets with subqueries	`ROLLUP`/`CUBE` with correlated sublinks	yes
Partitionwise join and aggregate	`enable_partitionwise_*` on `tp`, `tl`	via variants
Memoize, incremental sort	parameterised nested loops, presorted keys	via variants
Eager aggregation (if present in 19/20devel)	aggregates over joins	needs a GUC variant
`MERGE/SPLIT PARTITIONS` (if present)	partitioned tables with data	rule exists, may be unsupported

The commit miner (Section 5.1) will find the 20devel features we cannot list here; those get the highest priority because they have had the least testing.

6. Smaller generator upgrades

- **Plan-shape novelty feedback** (Query Plan Guidance, ICSE 2023 (verify)): hash the sequence of plan node types from `EXPLAIN (FORMAT JSON)` for a sample of queries; a new hash counts as new coverage. This steers the generator to unusual plans, which is where optimiser bugs hide.
- **Boundary literals from interval analysis** (L19-20): for every column the schema knows the type; generate literals at and around the boundaries of the value intervals that actually occur in the data (min, max, min-1, max+1, 0, -0, `NaN`, `infinity`, empty array, the longest text), not just random values.
- **Data-aware predicates** (constraint-based analysis, L27-28): generating `WHERE c = <value actually in the table>` instead of random constants makes predicates select rows, which exercises far more executor code. A tiny solver picks constants so that a predicate selects between 10 % and 90 % of rows.
- **Schema churn**: setup statements already create indexes, tables, views and statistics; we add `ANALYZE`, `VACUUM`, and changing table sizes between queries so the planner chooses different plans for the same query.

7. Triage: from candidate to accepted bug

This is how every candidate is handled. It is the part that earns the "real bug" marks, so it is automated as far as possible.

```
candidate -> reproduce x3 -> reduce -> classify -> bisect -> check release -> dedup -> report ->
```

1. **Reproduce** three times with `pgfuzz replay`. Flaky results are parked.
2. **Reduce** in two stages: first **grammar-tree reduction** (replace subtrees by the minimal-depth alternative of the same type, which keeps the query valid), then **ddmin** (Zeller, TSE 2002 (verify)) on statements and tokens. Typical goal: under 10 lines of SQL.
3. **Classify**: crash or assertion (always a bug), internal error with `XX000` (`elog(ERROR)` "should not happen" messages are almost always bugs), wrong result (needs the reasoning: which side is right and why, from the SQL standard or the docs).
4. **Bisect** with `git bisect run` and a script that builds and runs the reduced case. This names the commit that introduced the bug, which makes the report far more likely to be acted on quickly.
5. **Check the latest release** (and `REL_19_STABLE`): master-only bugs go to pgsql-hackers in the thread of the introducing commit; bugs in released versions go through the official bug report form (pgsql-bugs).
6. **Dedup** against the pgsql-bugs and pgsql-hackers archives and the open items page, by error message and by function name from the backtrace.
7. **Report** with a fixed template: version and commit, reduced SQL, expected vs actual output, backtrace for crashes, the bisected commit, and a one-sentence analysis. Short, reproducible, polite.
8. **Follow up**: answer questions the same day; test any proposed patch. The evidence we keep for the grade is the mailing-list thread URL, the bug number, and the fix commit with "Reported-by".

Course tie-in (L5, precision and recall). We record every candidate and its triage verdict. Oracle **precision** = real bugs / candidates, per oracle. This is a result in the paper, not just a process.

8. Evaluation upgrades

8.1 Research questions

- **RQ1 (validity and coverage).** How do the three arms compare in server validity rate, line and branch coverage, and feature/plan-shape diversity?
- **RQ2 (bugs).** How many unique crashes, assertion failures, internal errors and wrong results does each arm find, and how many are real?
- **RQ3 (time to detect known bugs).** On a benchmark of re-injected historical bugs, how fast does each arm find each bug?
- **RQ4 (LLM ablation).** What does the LLM contribute: grammar edits only vs seed synthesis only vs both, and at what token cost?

8.2 The re-injected bug benchmark (ground truth)

Picking 8 to 12 past fix commits for planner and executor bugs and **reverting** each one gives a server with a known bug. Every arm then runs for a fixed budget on each server; we record whether and when it finds the bug. This is the mutation-testing idea of L13 (inject faults, measure whether the test suite

detects them) applied to fuzzers, and it follows the Magma benchmark design (verify). It guarantees a meaningful RQ3 even if the number of new bugs is small.

Selection criteria: fix commit touches `src/backend/optimizer` or `executor`; has a regression test; the revert applies cleanly to our commit.

8.3 Statistics

Fuzzing is random, so single runs prove nothing (Klees et al., CCS 2018 (verify)). For each arm: **10 trials** of the same length, same schema, same server. We report medians with 95 % confidence intervals, the **Mann-Whitney U test** for significance and the **Vargha-Delaney** $\hat{A}_{12}$ effect size (Arcuri and Briand, ICSE 2011 (verify)). $\hat{A}_{12} = 0.5$ means no difference; above 0.71 is a large effect.

Budget, realistic for our machine: 3 arms x 10 trials x 1 hour, two workers in parallel, gives 15 hours wall-clock. Re-injection benchmark: 3 arms x 10 bugs x 3 trials x 20 minutes, about 30 hours with two workers, so we run it on a subset if time is short.

8.4 What the report must contain

The template grades five parts. This is where each one comes from, including the case with no new bug.

Graded part	What we write	If there is no new bug
Motivating example	One real query, the wrong behaviour, and measured numbers	A re-injected historical bug: the SQL, which arm found it, and the time
Workflow	The figure in Document 1: generator, executor, oracle, feedback. One tree used for generation, mutation, rewrite and reduction	Unchanged. The workflow is already built
Research questions	RQ1 to RQ4, each with a table and one observation	RQ2 states zero new bugs and the precision of each oracle
Related work	SQLsmith, SQLancer, Squirrel, SQLRight, Fuzz4All. Each paragraph ends with how we differ: the typed tree, the measured soundness, the validation gate	Unchanged
Threats to validity	Internal: randomness, 10 trials. External: one DBMS, one schema, one grammar fragment. Construct: signatures can merge two bugs or split one; we say so	Unchanged

Every number in these five parts comes from a run recorded under `results/`, never from an LLM summary.

9. Course lectures used (for the report and the viva)

Lecture	Concept	Where it appears
L2, L4, L5	Hybrid analysis, soundness, precision and recall	Commit mining directs a dynamic campaign. Generator soundness is the valid-statement rate; oracle soundness and precision come from triage (Section 0.3 and Section 7)
Hoare logic (check the lecture number)	Assertions on program behaviour	Each metamorphic rewrite is an assertion: the original and the rewritten query return the same multiset
L3, L21-22	Secure coding, information flow	Stretch: row-level security leak oracle (a query under RLS must never return a row the policy hides)
L6-L8	AST, CFG, IR, Clang/LLVM	Derivation trees as ASTs; coverage via gcov over the CFG; sanitizers
L13	Coverage, mutation testing	Coverage metrics; bug re-injection benchmark
L14-L16	Black-box and grey-box fuzzing, grammar fuzzing	The three arms; feature and plan-shape feedback; bandit weights
L19-20	Interval analysis	Boundary literal generation
L27-28	Constraint-based analysis	Data-aware predicates with target selectivity
L29-30	Type systems	Typed grammar: generation is proof search for $\Gamma \vdash e : T$

(Lecture numbers follow the course schedule; check the exact numbers against the syllabus before the report.)

10. Timeline and owners

Week	Dates	Venu (runs and new features)	Bhuvan (features and candidates)	Gowtham (experiment and paper)
1	15-21 Oct	sample loop is done (100% of 966); first 4 h campaign	metamorphic oracle (TLP, NoREC); start the feature list	commit miner, prompt design

Week	Dates	Venu (runs and new features)	Bhuvan (features and candidates)	Gowtham (experiment and paper)
2	22-28 Oct	campaigns every night; REL_19 build	rewrite oracle, reducer, bisect script; choose the 8-12 commits to re-inject	LLM loop with validation gate
	Meeting 2 (26-30 Oct)	first candidates and the validity numbers we already have	oracle demo	design slides
3	29 Oct - 4 Nov	evaluation runs (RQ1, RQ2)	triage every candidate, file reports	Grammar+LLM arm runs, diff coverage
	Meeting 3 (2-6 Nov)	results so far	bug status	ablation plan
4	5-11 Nov	re-injection benchmark (RQ3), ablation (RQ4)	follow up upstream	figures, paper draft
5	12-13 Nov	**presentation 12 Nov**		**paper and repo 13 Nov**

The names above are the split from Section 0.5. The earlier build split (harness, generators, evaluation) is how the repository was written; it is not a different team. The rule is that **at least one campaign is running at every moment from week 1 on**, because bug finding is a function of CPU hours.

11. Risks and fallbacks

Risk	Likelihood	Fallback
Grammar validity stays low	The latest sample is 100% of 966 statements. New rules will drop it again.	Keep lowering the weight of failing alternatives; the guided mutation still yields valid queries. Retune the same way when a new feature rule lands
No new bug before the deadline	medium	File in week 2 anyway. RQ1, RQ3 and RQ4, and all five graded report sections, are already complete. We do not treat that as full marks on the 50 % implementation component
Upstream does not respond in time	medium	report early (week 2); a confirmed reply on the list counts as evidence even before the fix

Risk	Likelihood	Fallback
Bug turns out to be a duplicate	medium	dedup step before reporting; duplicates still validate the fuzzer in the paper
LLM proposals are mostly invalid	medium	validation gate rejects them; the acceptance rate itself is a result
WSL or machine instability	low	the harness restarts the server; campaigns are resumable per trial
False positives in wrong-result oracles	medium	re-run, cross-check with planner variants, manual reasoning before any report

12. References (all: verify before citing)

- M. Rigger, Z. Su. *Finding Bugs in Database Systems via Query Partitioning* (TLP). OOPSLA 2020.
- M. Rigger, Z. Su. *Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction* (NoREC). ESEC/FSE 2020.
- M. Rigger, Z. Su. *Testing Database Engines via Pivoted Query Synthesis* (PQS). OSDI 2020.
- J. Ba, M. Rigger. *Testing Database Engines via Query Plan Guidance*. ICSE 2023.
- J. Ba, M. Rigger. *CERT: Finding Performance Issues in Database Systems Through the Lens of Cardinality Estimation*. ICSE 2024.
- Z. Jiang, Z. Su. *Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation* (EET). OSDI 2024.
- R. Zhong et al. *Squirrel: Testing Database Management Systems with Language Validity and Coverage Feedback*. CCS 2020.
- Y. Liang et al. *Detecting Logical Bugs of DBMS with Coverage-based Guidance* (SQLRight). USENIX Security 2022.
- C. S. Xia et al. *Fuzz4All: Universal Fuzzing with Large Language Models*. ICSE 2024.
- M. Böhme et al. *Directed Greybox Fuzzing* (AFLGo). CCS 2017.
- A. Hazimeh, A. Herrera, M. Payer. *Magma: A Ground-Truth Fuzzing Benchmark*. SIGMETRICS 2021.
- G. Klees et al. *Evaluating Fuzz Testing*. CCS 2018.
- A. Arcuri, L. Briand. *A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering*. ICSE 2011.
- A. Zeller, R. Hildebrandt. *Simplifying and Isolating Failure-Inducing Input* (ddmin). TSE 2002.
- A. Seltenreich. SQLsmith (tool, GitHub).

13. The next concrete step

(you) From `src/`, run:

```
python -m pgfuzz -q sample configs/local.yaml -n 300 --execute `
    --seed 1 --out ../results/sample1
```

and paste the full output. I tune the grammar from it; then you start the 4-hour `hunt.yaml` campaign, and I build the metamorphic oracle and triage tools while it runs.